

MEAM 6200 Project 1.4 Report

Team 10: Jason Chen, Smrithi Seramalanna, Zhihan Wang, Sumukh Shrote March 26, 2026

I. INTRODUCTION AND SYSTEM OVERVIEW

The goal of the MEAM 620 Crazyflie labs was to transition the developed Project 1 algorithms from simulation to a physical quadrotor flying in a motion capture environment. In Lab 1, the position controller and trajectory generator were tuned to execute basic maneuvers, including a stable hover, step responses, and a box pattern. In Lab 2, the A* path planner was utilized to navigate a maze with physical obstacles, chaining sequential start and goal positions across three map configurations. Upon completion, the system successfully demonstrated stable hovering, accurate trajectory tracking, collision-free path planning, and minimum-jerk trajectories that the physical hardware could feasibly execute without crashing.

The primary hardware platform was the Bitcraze Crazyflie 2.0 quadrotor—a lightweight 27g drone equipped with an onboard STM32F405 microcontroller, an MPU-9250 IMU, and a 2.4 GHz radio. External sensing was provided by the Vicon motion capture system, which utilized a network of infrared cameras to track passive reflective markers attached to the Crazyflie, delivering highly accurate 6-DoF position and orientation state estimation. All high-level computation and outer-loop control were executed on a lab desktop computer running Ubuntu and ROS, which contained the custom Python control and planning scripts.

Information flow between the hardware components was strictly divided to manage latency. The Vicon system sent continuous 6-DoF pose updates unidirectionally to the lab desktop computer via ROS topics to serve as feedback for the outer-loop position controller. In response, the lab desktop sent specific high-level control commands—specifically, desired attitude quaternions (cmd_q) and desired total thrust (cmd_{thrust})—over a Crazyradio USB dongle directly to the quadrotor. Due to radio communication delays, no direct motor torques or individual motor speeds were sent from the desktop. Instead, the Crazyflie’s onboard firmware received the attitude and thrust commands and handled the high-frequency motor mixing (≈ 500 Hz) using its internal attitude controller and onboard IMU data. Finally, all system data, including Vicon odometry, planned trajectory waypoints, and commanded outputs, were actively sent to the desktop and recorded into ROS .bag files for post-flight analysis.

II. CONTROLLER

A. Position Controller Formulation

Our position controller relies on a Proportional-Derivative (PD) architecture to track the desired trajectory. We first define the position error $\mathbf{e}_r$ and velocity error $\mathbf{e}_v$ as the difference between our actual state and the desired state:

$$\mathbf{e}_r = \mathbf{r} - \mathbf{r}_{des}, \quad \mathbf{e}_v = \mathbf{v} - \mathbf{v}_{des} \quad (1)$$

Using these error terms, the commanded acceleration $\ddot{\mathbf{r}}_{cmd}$ is calculated by applying our proportional (K_p) and derivative (K_d) gain matrices:

$$\ddot{\mathbf{r}}_{cmd} = \ddot{\mathbf{r}}_{des} - K_p \mathbf{e}_r - K_d \mathbf{e}_v \quad (2)$$

Finally, the desired force vector $\mathbf{F}_{des}$ required to achieve this acceleration, compensating for the acceleration of gravity g and the quadrotor’s mass m , is defined as:

$$\mathbf{F}_{des} = m(\ddot{\mathbf{r}}_{cmd} + g\hat{\mathbf{z}}_W) \quad (3)$$

The total commanded thrust is then extracted by projecting this desired force onto the quadrotor’s current body z -axis.

B. Control Gains and Tuning

To account for the differences between the ideal simulation and the physical environment, significant adjustments were required across all control gains. Our original gains tuned in the simulation environment were:

- Position Proportional (K_p): $\text{diag}(15, 15, 22) \text{ s}^{-2}$
- Position Derivative (K_d): $\text{diag}(8, 8, 13) \text{ s}^{-1}$
- Attitude Proportional (K_R): $\text{diag}(3000, 3000, 150) \text{ s}^{-2}$
- Attitude Derivative (K_ω): $\text{diag}(300, 300, 50) \text{ s}^{-1}$

However, the most recently tuned control gains used for our in-person hardware flights are drastically lower:

- Position Proportional (K_p): $\text{diag}(6, 6, 15) \text{ s}^{-2}$
- Position Derivative (K_d): $\text{diag}(4, 4, 9) \text{ s}^{-1}$
- Attitude Proportional (K_R): $\text{diag}(300, 300, 15) \text{ s}^{-2}$
- Attitude Derivative (K_ω): $\text{diag}(30, 30, 5) \text{ s}^{-1}$

The position proportional gain (K_p) acted as a virtual spring, dictating how aggressively the quadrotor pulled itself toward the target position in the x, y , and z axes. The derivative gain (K_d) acted as a virtual damper, resisting velocity errors to prevent the vehicle from overshooting the target.

When transitioning to hardware, we were recommended to start with lower gains. The position gains (K_p and K_d) were roughly halved, and the attitude gains (K_R and K_ω) were reduced by a full order of magnitude (a factor of 10). Plausible reasons for these necessary reductions stem from unmodeled real-world dynamics. In simulation, the quadrotor is treated as a perfect rigid body with instantaneous motor responses and zero latency. On the physical hardware, commanding high attitude gains requests aggressive moments that can easily saturate the physical motors. Combined with the 10–20 ms latency in the Vicon-to-radio communication loop, these aggressively fast commands lead to overcorrection and instability. Lowering the gains appropriately damped out these real-world oscillations, yielding a stable system that still maintained rapid and reliable tracking.

C. Attitude Control Architecture

While our `SE3Control` class calculated the desired moment (u_2) using angular error (e_R) and angular velocity error (e_ω), this moment was not directly sent to the motors during hardware flights. Instead, the script extracted the desired rotation matrix R_{des} and converted it into a quaternion (`cmd_q`).

This occurred because attitude dynamics were highly non-linear and changed very rapidly. The control loop running on the desktop computer relied on Vicon data and radio communication, which introduced a delay of roughly 10–20 ms. Sending direct motor moments over this latent connection would result in an unstable system. Instead, the desired attitude quaternion (`cmd_q`) and total thrust (`cmd_thrust`) were sent over the radio to the Crazyflie. The Crazyflie’s onboard microcontroller then utilized its own high-frequency (≈ 500 Hz) internal attitude controller alongside its onboard IMU to calculate and command the individual motor mixing speeds.

D. Controller Performance

To evaluate the dynamic response of our tuned controller, a brief section of one of our first session trajectory experiments was isolated: a lateral (y -axis) step response. The controller performance was assessed based on steady-state error, damping ratio (ζ), rise time, and settling time.

When subjected to a step command, the vehicle exhibited characteristics of a slightly underdamped system. The quadrotor demonstrated

a swift rise time of approximately 0.584 s, but experienced a minor overshoot of 14.4%, which corresponds to an estimated damping ratio (ζ) of 0.525. It then stabilized, achieving a settling time of roughly 1.498 s to enter the 5% error band. This aligns with the heuristics-based tuning approach recommended in the lab manual, which suggests balancing a rapid initial response with manageable overshoot.

Fig. 1 illustrates these transient characteristics over time. As seen in the plot, there is a slight steady-state offset, resulting in a post-step average error of roughly 0.024 m. This steady-state error is an expected characteristic of our PD position controller. Because we do not utilize an integral term (K_i) in our position control formulation, the controller cannot perfectly eliminate accumulated errors caused by unmodeled constant disturbances, such as slight imbalances in battery weight distribution or aerodynamic ground effect.

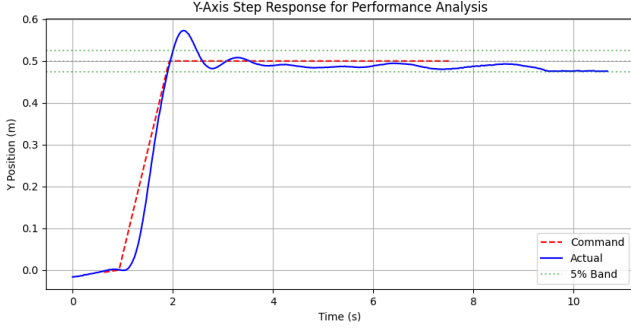


Fig. 1. Y-axis position vs. time during a step trajectory experiment, illustrating transient performance characteristics such as rise time, overshoot, and settling time. A minor steady-state offset is visible due to the lack of an integral control term.

III. TRAJECTORY GENERATOR

A. Waypoint Generation and Post-Processing

In order to generate waypoints given a known map, the space was discretized into a regular grid and graph nodes were attached to each voxel. Then, A* algorithm was implemented to this grid to generate a reasonable and collision free path. We use the Euclidean distance from the current position to the goal as the heuristic equation, as follows.

$$h(v) = \sqrt{(x - x_g)^2 + (y - y_g)^2 + (z - z_g)^2} \quad (4)$$

However, this path should not be used directly for trajectory generation as many of the generated waypoints were unnecessary and could be pruned. Additionally, they may negatively affect trajectory generation and cause collisions between the drone and obstacles. For example, for many nearly straight paths, redundant waypoints were generated. This made the final trajectory overly oscillatory and caused unstable flight of the drone. Also, A* generated too many dense waypoints near the start and goal points. These waypoints may prevent the drone from stopping stably at the goal. To avoid these problems, the following two processing methods were applied to the generated waypoints.

a) 1: First, “collinear points” were removed. Trigonometric relations were used to determine whether three points were collinear. If they were collinear, the middle point was removed; otherwise, it was kept as a turning point. However, this may create a case where only two or three points remain on a straight line, which can cause the trajectory to touch obstacles. To avoid this, an extra constraint was added: if removing the middle point made the distance between the other two points greater than $maxdist_ptp$, then the middle point was kept regardless. By adjusting $maxdist_ptp$, the number of waypoints on straight segments were limited. Upon repeated trial and error in simulation tests, the optimal $maxdist_ptp$ was found to be 1 m to generate smooth and non oscillatory trajectories.

The collinearity test uses the cross-product criterion. For three consecutive points, two direction vectors are formed: $\mathbf{V}_a = \mathbf{p}_i - \mathbf{p}_{keep}$ and $\mathbf{V}_b = \mathbf{p}_{i+1} - \mathbf{p}_i$. The sine of the angle between them is computed

as:

$$\sin \theta = \frac{\|\mathbf{V}_a \times \mathbf{V}_b\|}{\|\mathbf{V}_a\| \|\mathbf{V}_b\|} \quad (5)$$

A point is considered collinear and removed if $\sin \theta < \epsilon$, where $\epsilon = 10^{-6}$ was chosen as the threshold. This value was selected to be small enough to catch near-perfectly straight segments while retaining all genuine direction changes.

The occupancy grid used for A* was discretized at a resolution of $\Delta = 0.12$ m in all three axes, with obstacle cells inflated by a safety margin of $m = 0.25$ m. The resolution controls the granularity of the search where finer resolution produces smoother paths but increases computation time. The margin ensures the planned path maintains a minimum clearance from obstacles, accounting for the physical size of the Crazyflie and small positional errors during flight.

b) 2: In addition, a distance filter was also applied to all waypoints. When the distance between two points is smaller than d_{min} , the second point was removed. This filter prevented too many waypoints from accumulating near the start point, the goal point, and corners, making the trajectory more reasonable. d_{min} was set to 0.4 m to achieve the best result, as determined in simulation.

B. Time Allocation Between Waypoints

Segment durations were allocated proportionally to the Euclidean distance between successive waypoints at a fixed cruise speed v :

$$T_k = \frac{\|\mathbf{p}_{k+1} - \mathbf{p}_k\|_2}{v} \quad (6)$$

where T_k is the duration of segment k , $\mathbf{p}_k$ and $\mathbf{p}_{k+1}$ are successive waypoint positions, and $v = 0.3 \text{ m s}^{-1}$ is the cruise speed used during hardware flights. This proportional allocation ensures that longer segments are given more time, keeping the commanded velocity bounded near v throughout the trajectory and preventing unrealistically fast commands on short segments. Compared to a fixed-time allocation, this approach produces more consistent velocity profiles and reduces peak acceleration demands on the controller.

C. Trajectory Generation Formulation

A minimum jerk trajectory coupled with a fifth order polynomial was used for trajectory generation. Continuous position, velocity, acceleration, jerk, and snap are enforced at each waypoint. The fifth order polynomial which represents each trajectory segment is as follows:

$$p_k(t) = a_{k,0} + a_{k,1}t + a_{k,2}t^2 + a_{k,3}t^3 + a_{k,4}t^4 + a_{k,5}t^5 \quad (7)$$

where $p_k(t) \in \mathbb{R}^3$ is the position along one axis at time t ; k is the segment index; T_k is the duration of segment k ; and $a_{k,0}, \dots, a_{k,5}$ are the polynomial coefficients solved per axis. Its derivatives, giving velocity, acceleration, jerk, and snap respectively, are:

$$\dot{p}_k(t) = a_{k,1} + 2a_{k,2}t + 3a_{k,3}t^2 + 4a_{k,4}t^3 + 5a_{k,5}t^4 \quad (8)$$

$$\ddot{p}_k(t) = 2a_{k,2} + 6a_{k,3}t + 12a_{k,4}t^2 + 20a_{k,5}t^3 \quad (9)$$

$$p_k^{(3)}(t) = 6a_{k,3} + 24a_{k,4}t + 60a_{k,5}t^2 \quad (10)$$

$$p_k^{(4)}(t) = 24a_{k,4} + 120a_{k,5}t \quad (11)$$

The boundary conditions enforce zero velocity and acceleration at the start and end of the full trajectory:

$$p_0(0) = \mathbf{p}_0, \quad \dot{p}_0(0) = 0, \quad \ddot{p}_0(0) = 0 \quad (12)$$

$$p_{m-1}(T_{m-1}) = \mathbf{p}_{N-1}, \quad \dot{p}_{m-1}(T_{m-1}) = 0, \quad \ddot{p}_{m-1}(T_{m-1}) = 0 \quad (13)$$

where $\mathbf{p}_0$ and $\mathbf{p}_{N-1}$ are the start and goal positions, and m is the total number of segments.

At each interior waypoint $\mathbf{p}_{k+1}$, position continuity is enforced from both sides:

$$p_k(T_k) = \mathbf{p}_{k+1} \quad (14)$$

$$p_{k+1}(0) = \mathbf{p}_{k+1} \quad (15)$$

Finally, derivative continuity is enforced across segment boundaries up to snap, ensuring C^4 smoothness throughout the trajectory:

$$\dot{p}_k(T_k) = \dot{p}_{k+1}(0) \quad (16)$$

$$\ddot{p}_k(T_k) = \ddot{p}_{k+1}(0) \quad (17)$$

$$p_k^{(3)}(T_k) = p_{k+1}^{(3)}(0) \quad (18)$$

$$p_k^{(4)}(T_k) = p_{k+1}^{(4)}(0) \quad (19)$$

D. Smoothness/Feasibility of Planned Trajectory

The smoothness and feasibility of our trajectory generator are actually very good. For the test maze map, we not only found a shorter path, but the processed waypoints are also very reasonable. The minimum jerk objective, together with the constraints that ensure continuity from position all the way to snap, makes the trajectory highly smooth. The trajectory neither collides with obstacles nor produces unnecessary “spikes” or large angle curves. This ensures the feasibility of the flight.

Fig. 2 displays the plots of position and velocity over time, which show that the actual position and velocity of the drone closely match the desired values in all three axes (x, y, z). Minor deviations appear when there are rapid changes. Moreover, the velocity does not show large fluctuations, which reduces the burden on the controller and greatly improves the feasibility of executing the trajectory. From the orientation vs time plots in Fig. 3, it can be seen that the actual angular velocity and quaternion curves have a high degree of overlap with the desired curves. These observations indicate that the controller can follow the generated trajectory accurately and demonstrates strong feasibility.

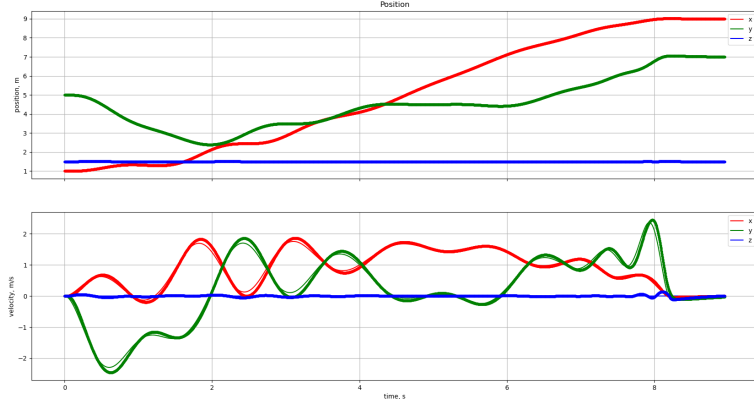


Fig. 2. Position vs Time / Velocity vs Time

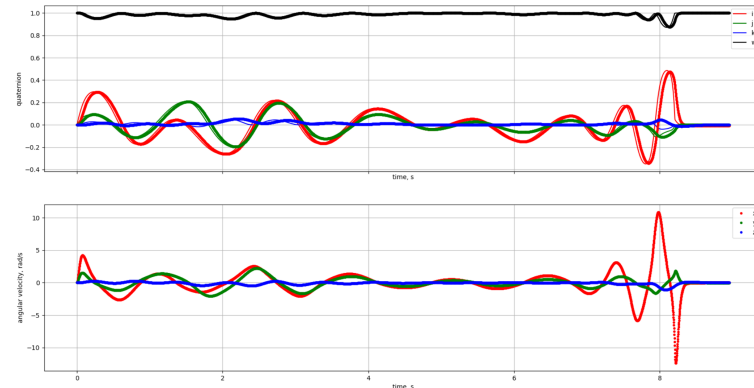


Fig. 3. Orientation vs Time

IV. MAZE FLIGHT EXPERIMENTS

Three maze flights were performed with chained start/goal positions: Maze 1 from $(-1.75, -1.5, 0.75)$ to $(-0.712, 1.4, 0.75)$ m, Maze 2 from $(-0.712, 1.4, 0.75)$ to $(2.2, 0.75, 0.75)$ m, and Maze 3 the full reverse from $(2.2, 0.75, 0.75)$ to $(-1.75, -1.5, 0.75)$ m. All three completed without collision. Fig. 4, Fig. 5, and Fig. 6 show the 3D obstacle environment with dense A* path (red), sparse waypoints (purple), planned trajectory (black), actual flight path (blue), start (green circle), and goal (red circle) for each of the 3 maze runs.

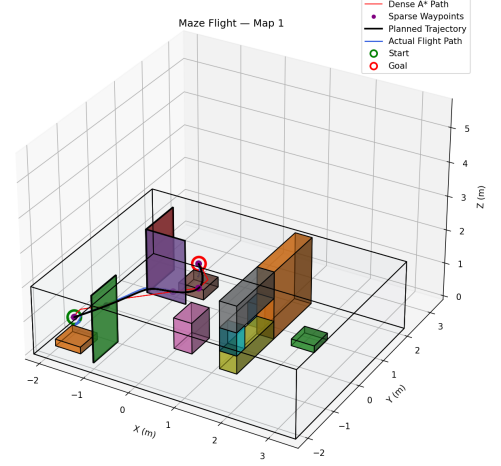


Fig. 4. Maze 1: World obstacles, waypoints, planned trajectory, and actual flight path from $(-1.75, -1.5, 0.75)$ to $(-0.712, 1.4, 0.75)$ m.

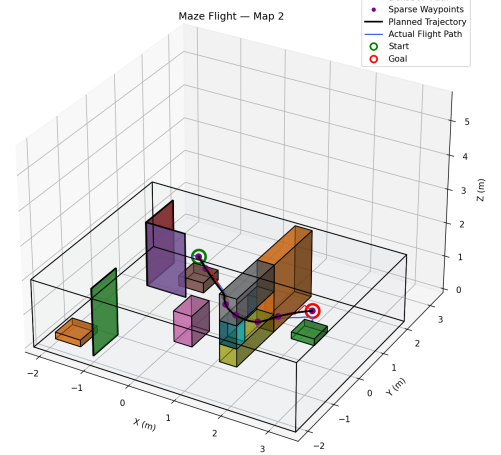


Fig. 5. Maze 2: World obstacles, waypoints, planned trajectory, and actual flight path from $(-0.712, 1.4, 0.75)$ to $(2.2, 0.75, 0.75)$ m.

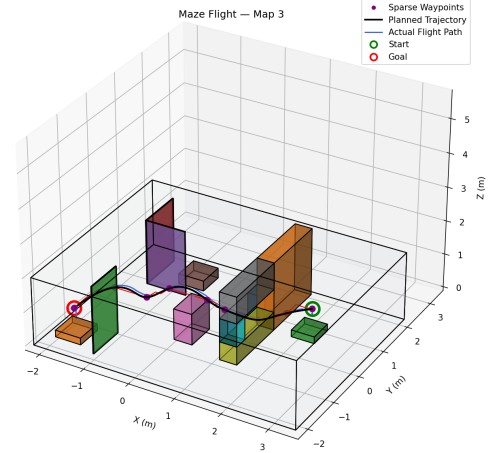


Fig. 6. Maze 3: World obstacles, waypoints, planned trajectory, and actual flight path from $(2.2, 0.75, 0.75)$ to $(-1.75, -1.5, 0.75)$ m.

Fig. 7 shows position and velocity vs. time for Maze 1 (flight time ≈ 11 s). Altitude holds near 0.75 m with deviations below 0.15 m.

Lateral velocities peak near $\pm 0.3 \text{ m s}^{-1}$ and return smoothly to zero at the goal, consistent with the minimum-snap boundary conditions.

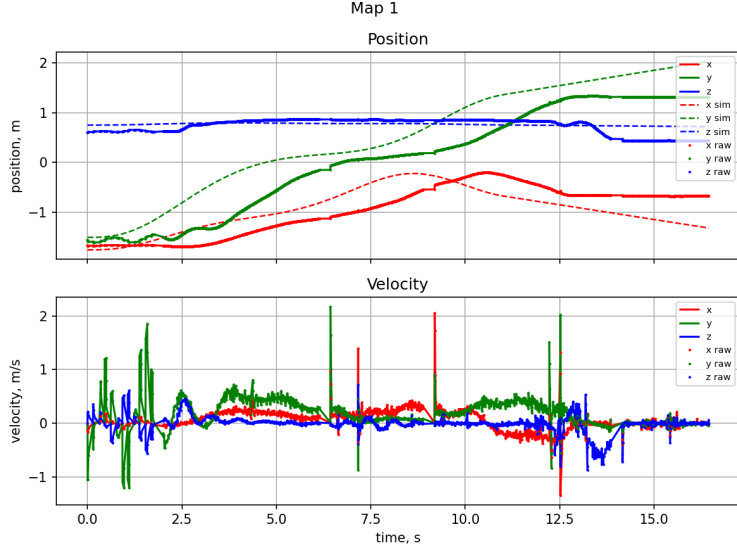


Fig. 7. Position (top) and velocity (bottom) vs. time, Maze 1. Solid: actual (Vicon). Dashed: planned. Dots: raw readings.

A. Tracking Error

Tracking error is computed at each timestep by comparing the actual position from Vicon odometry against the desired position from the trajectory flat outputs. Since these are sampled at different rates, the desired trajectory is interpolated onto the odometry timestamps using linear interpolation.

These are collapsed into a single 3D Euclidean norm and the per-axis errors are defined as:

$$e_x = x_{\text{act}} - x_{\text{des}}, \quad e_y = y_{\text{act}} - y_{\text{des}}, \quad e_z = z_{\text{act}} - z_{\text{des}} \quad (20)$$

$$e_{3D} = \sqrt{e_x^2 + e_y^2 + e_z^2} \quad (21)$$

where $x_{\text{act}}, y_{\text{act}}, z_{\text{act}}$ are the actual vehicle positions measured by Vicon odometry; $x_{\text{des}}, y_{\text{des}}, z_{\text{des}}$ are the desired positions commanded by the trajectory generator flat outputs; e_x, e_y, e_z are the signed per-axis position errors; and e_{3D} is the 3D Euclidean norm representing the straight-line distance between the actual and desired position at each instant. Performance is summarised using Root Mean Square Error (RMSE), computed over the full flight duration for each axis:

$$\text{RMSE}_x = \sqrt{\frac{1}{N} \sum_{i=1}^N e_{x,i}^2}, \quad \text{RMSE}_{3D} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_{3D,i}^2} \quad (22)$$

Fig. 8 shows e_{3D} for all mazes; RMSE results are summarised in Table I. All mazes share a characteristic signature: error rises from $\approx 0.15 \text{ m}$ to a peak of 0.9 m to 1.1 m within the first 2–3 s (initial acceleration phase), then plateaus for the remainder of the flight. The z -axis RMSE is consistently the lowest across all three mazes (0.08 m to 0.23 m), confirming effective altitude hold. This is partly due to the inherent controllability of altitude on a quadrotor, and partly because all three maze start and goal positions share the same height of 0.75 m , meaning the trajectory demands minimal z variation throughout the flight. The dominant error is lateral (x/y), with RMSE_x and RMSE_y ranging from 0.34 m to 0.57 m .

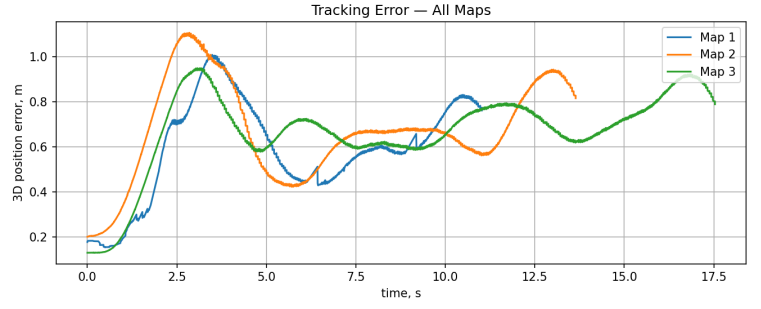


Fig. 8. 3D tracking error vs. time, all three mazes.

TABLE I

Maze	RMSE _x (m)	RMSE _y (m)	RMSE _z (m)	RMSE _{3D} (m)	Time (s)
1	0.339	0.575	0.082	0.672	11.0
2	0.510	0.463	0.156	0.706	13.6
3	0.548	0.354	0.231	0.692	17.5

Lateral (x/y) RMSE is substantially larger than z RMSE across all three mazes for three main reasons. First, the quadrotor produced lateral force by tilting its body as thrust was redirected horizontally only when the vehicle pitched or rolled. Following a lateral trajectory command therefore required the attitude to change quickly, which was limited by the attitude controller’s response time. Additionally, when the trajectory demanded a faster lateral acceleration than the vehicle could tilt to achieve, the drone fell behind. The z -axis did not have this problem because altitude was controlled directly by increasing total thrust, which the rotors could do without tilting. Second, the position gains were reduced for hardware from 15 N/m to 6 N/m to prevent noise-driven oscillations. Lower gains meant the controller corrected position errors more slowly, allowing the drone to lag further behind the planned trajectory before pulling back toward it. Third, Vicon measurement noise and radio communication latency meant the correction the controller computes was always based on a slightly outdated position. By the time the motor command was applied, the desired position had already moved further along the trajectory. To plan safe trajectories given these errors, the obstacle inflation margin should be at least as large as the observed RMSE_{3D} of 0.67 m to 0.71 m , substantially exceeding the current margin of 0.25 m , so a margin of 0.5 to 0.6 m would be recommended for safe flight.

B. Aggressiveness and Improvements

The cruise speed of 0.3 m s^{-1} was chosen conservatively after finding that the initial allocation of 1.5 m s^{-1} was too fast for reliable hardware tracking as the vehicle could not keep up with the aggressive trajectory commands, producing large tracking errors and risking collision. At 0.3 m s^{-1} the observed RMSE_{3D} ranged from 0.67 m to 0.71 m across all three mazes, which was more manageable but still exceeded the obstacle margin of 0.25 m . Increasing the quadrotor’s speed may be feasible with a larger margin and curvature-aware allocation, meaning it would go faster on straights but slower at corners without compromising safety.

Key improvements: (1) curvature-aware time allocation to exploit straight segments at higher speeds while slowing near tight corners; (2) increased margin to 0.5 m to match observed errors; (3) notch filtering on Vicon estimates to allow higher position gains.

Given one more session, we would set start and goal positions at different altitudes to stress-test the z -axis tracking performance. In the current experiments, all start and goal heights were fixed at 0.75 m , meaning the trajectory demanded very little vertical motion and the z controller was never meaningfully challenged. Assigning start and goal positions at different heights (e.g. 0.3 m to 1.5 m) would force the drone to climb or descend through the maze, revealing whether the altitude controller maintains the same low error observed in lateral-only flight or degrades under combined lateral and vertical demands.